

GLKVM Signal Bridge — Documentation

Conceptual and architectural notes on this project. The README is the quick-reference; this doc is the "I haven't touched this in months, how does it work again" reference.

Table of contents

1. [What the system does](#)
 2. [System diagram](#)
 3. [The four data flows](#)
 4. [Component-by-component](#)
 5. [Why this architecture](#)
 6. [Operational gotchas](#)
 7. [Extending the system](#)
 8. [Glossary](#)
-

What the system does

You have two computers you sometimes need to control from your phone or remote location:

- A **MacBook Air** (your daily work laptop)
- A **Windows laptop** (your other work environment)

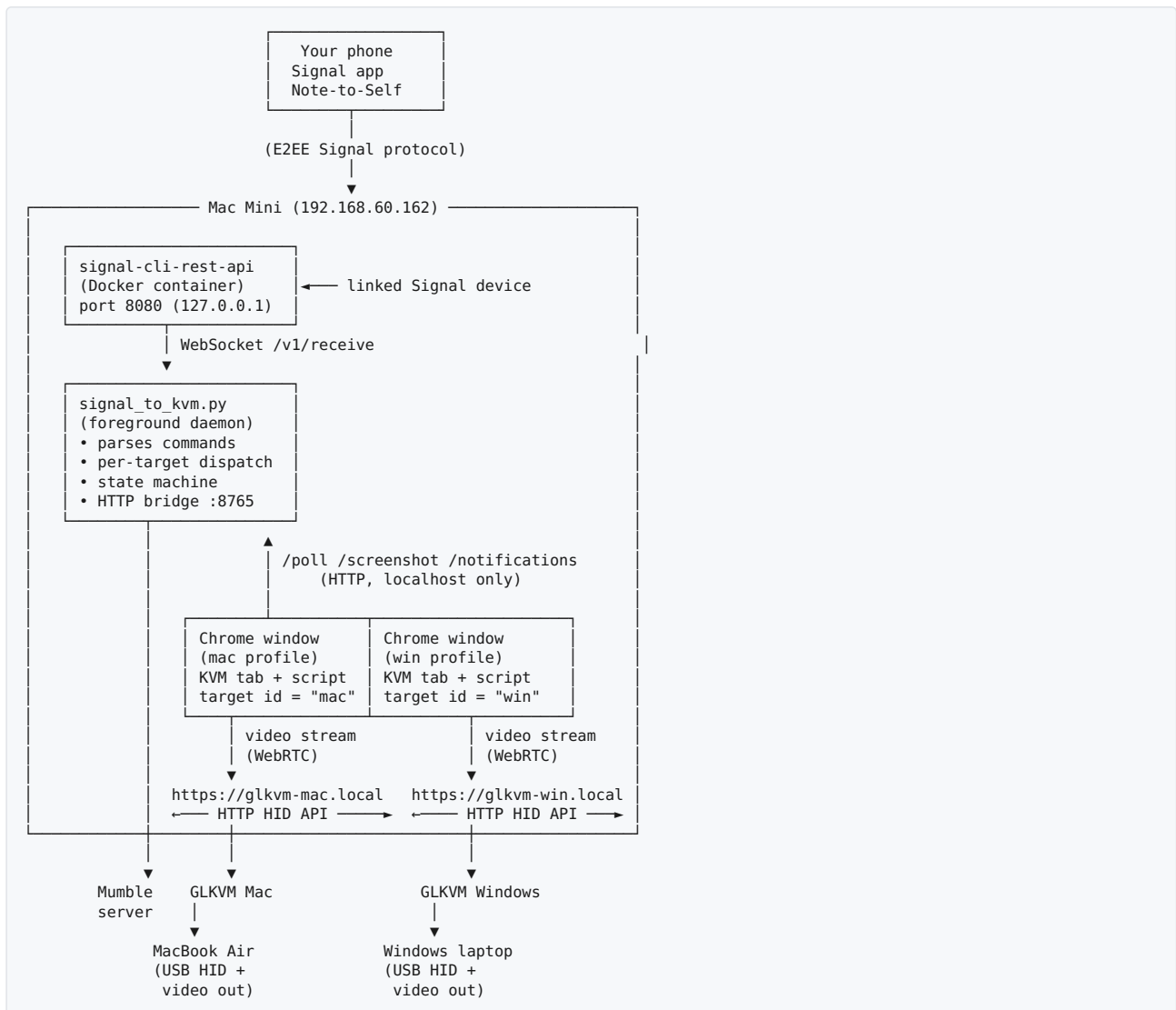
Each is connected to its own **GL.iNet Comet PRO KVM** — a small device that emulates a USB keyboard/mouse to the host PC and streams the host's video over the network. The KVM is the only thing on the network that can "type" into those laptops.

This project lets you send commands to those KVMs via **Signal messages to yourself** (Note to Self). The commands cover the things you'd actually want to do remotely:

- Open a Teams chat with someone by name
- Type a message into the focused chat box and confirm before sending
- Set your Teams status (online/busy/away)
- Take screenshots of either laptop's screen
- Get alerted when something turns red on screen (e.g., a Teams unread badge)
- Toggle the KVM's built-in mouse jiggler
- Cycle the keyboard input layout

All of this runs on your **Mac Mini at home** — always on, static IP, single control point for the whole setup.

System diagram



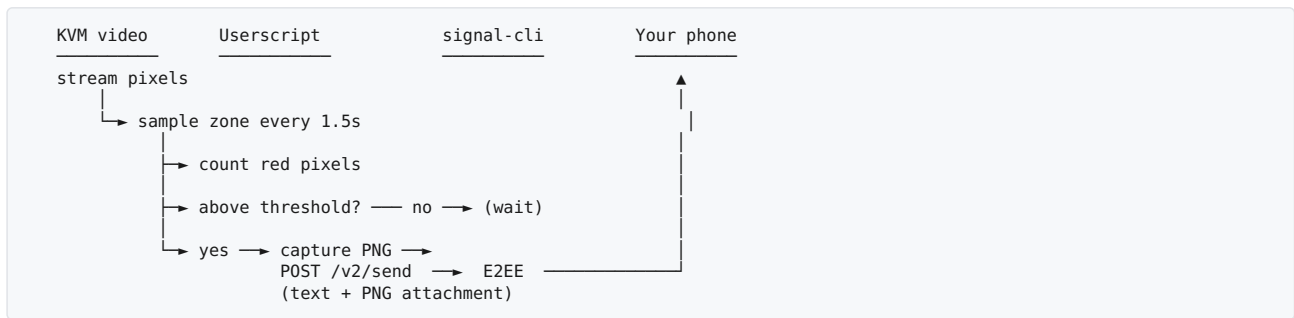
Three things to notice:

1. **Two paths to each KVM**, from the same Mac Mini: the daemon talks HTTP for HID events (clicks, keystrokes), and the Chrome tab talks WebRTC for the video stream. They're independent.
2. **The userscript is a bridge**, not a tool you operate. Its panel exists for setup/diagnostics, but in normal use, you don't interact with it — it polls the daemon, fulfils screenshot requests, and silently watches for red pixels.
3. **All traffic on 127.0.0.1** except the HID API calls and KVM hostnames. Nothing the bridge does is exposed to the LAN unless you change the bind address. Everything between daemon, Signal API, and userscripts is localhost.

The four data flows

Flow 1: Watcher alert (red pixel → Signal)

This is the "passive" mode: the userscript watches the KVM video stream and alerts when it sees red.

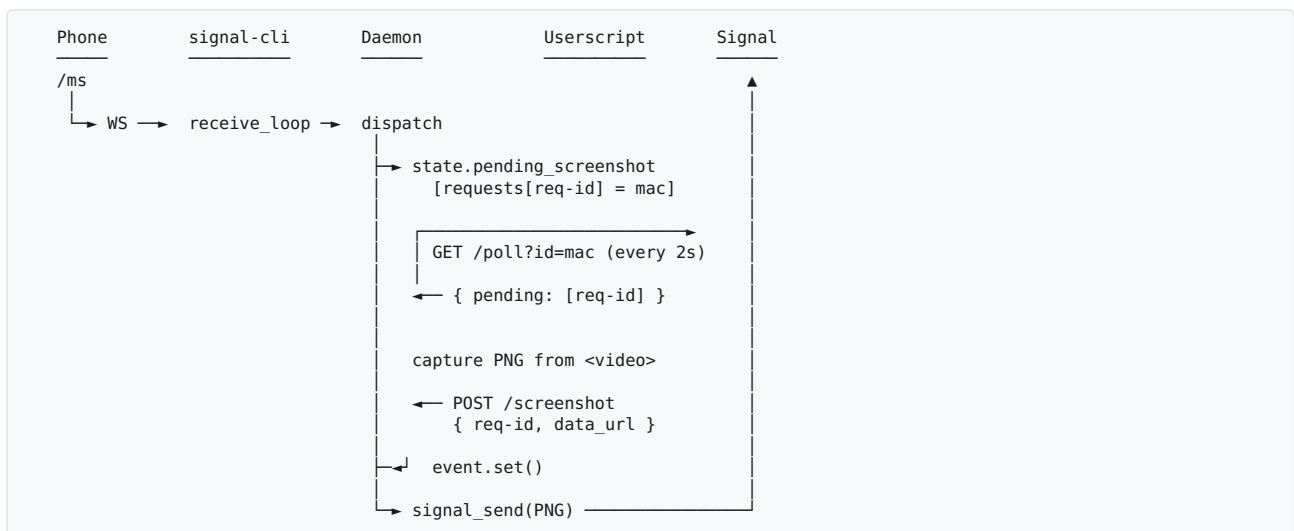


This flow doesn't touch the daemon at all. The userscript calls Signal's HTTP API directly. The reason: less moving parts, lower latency, and the daemon would just forward it anyway.

The userscript checks the daemon's `/poll` response for the `notifications_enabled` flag — if you've sent `/dt` from Signal, the userscript stops sampling. That's the daemon's only involvement here.

Flow 2: Simple command (e.g., `make-screenshot`)

This is the "fetch and report" pattern: command arrives, daemon needs the userscript's help to get a screenshot, daemon posts the result.

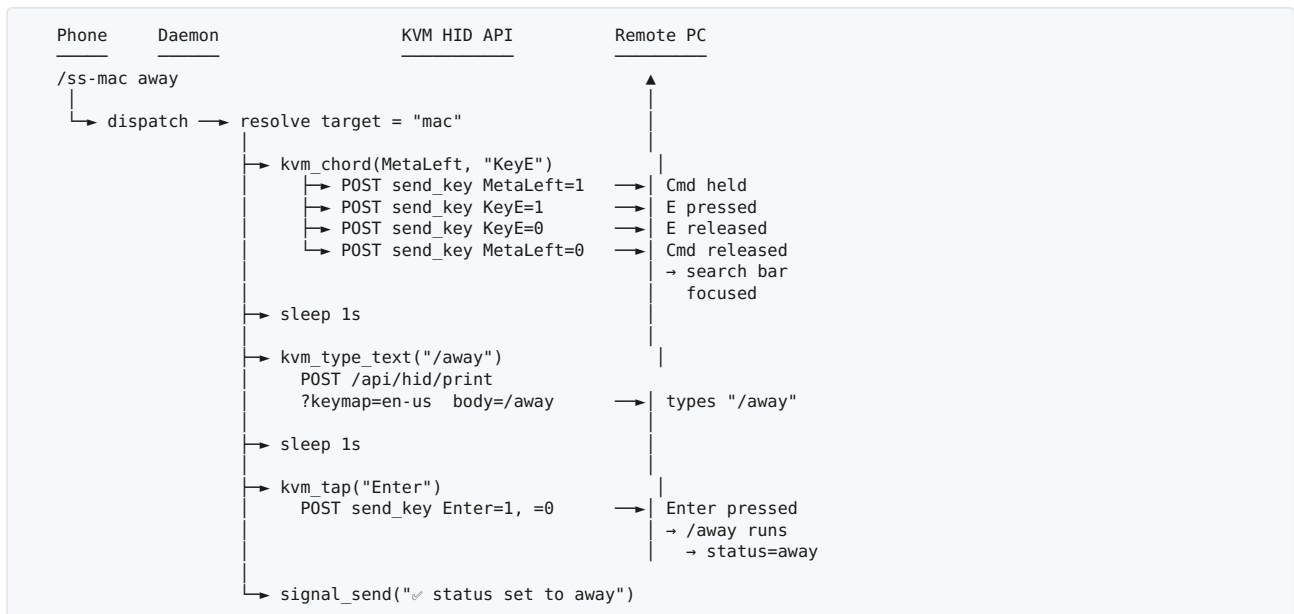


Key thing to notice: **the daemon doesn't know how to take screenshots**. It only knows how to ask. Because the screenshots come from the WebRTC video stream that lives in a browser, only something running in the browser can grab the pixels. The userscript is the daemon's "eyes."

The polling latency (~2s max) is why simple commands feel snappier than this one — they don't wait for a poll cycle.

Flow 3: Action command (e.g., `set-status away`)

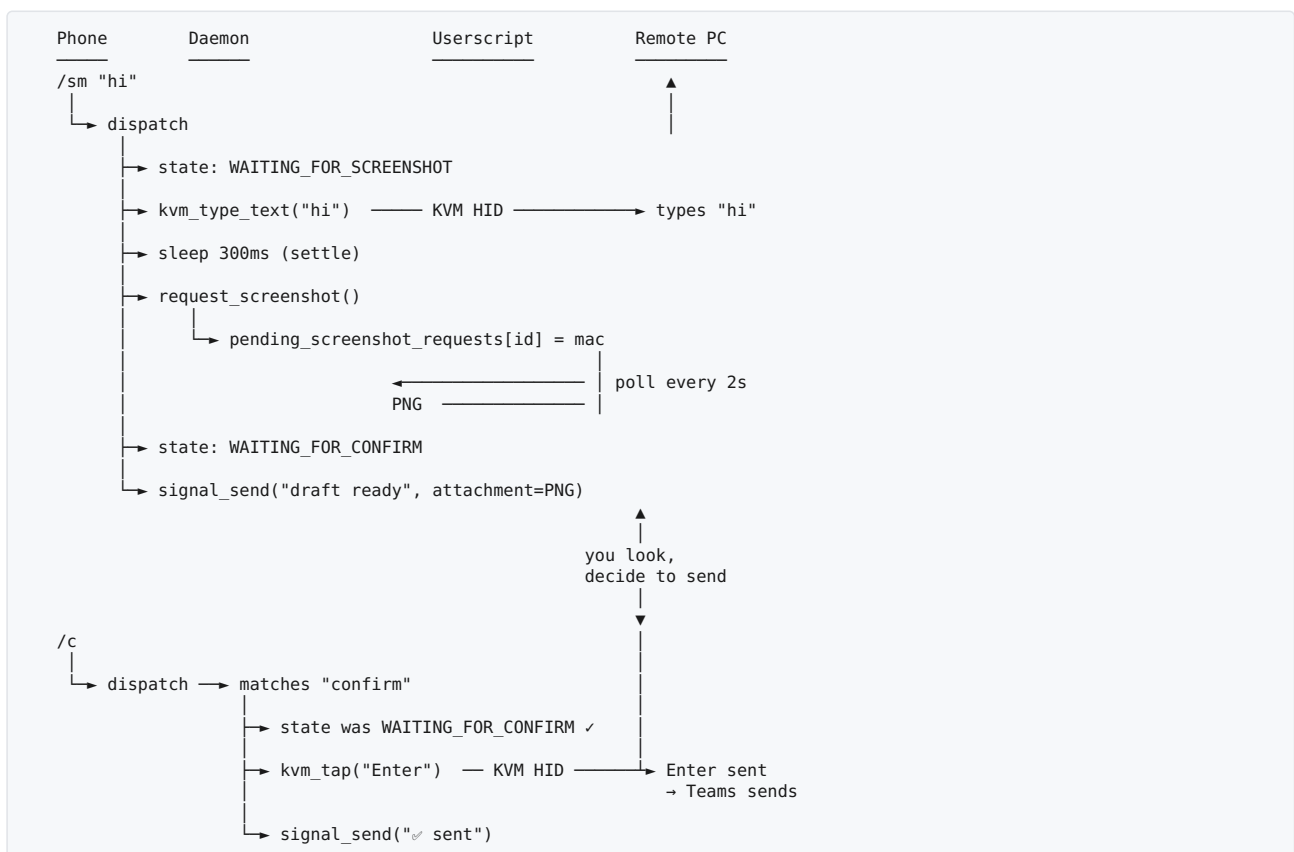
The "type and react" pattern: command arrives, daemon issues keystrokes via the KVM HID API, no screenshot needed.



Each step is a separate HTTP call to the KVM. The sleeps between steps are tuned so Teams has time to react before the next keystroke. Too short and the next keystroke arrives before the UI is ready; too long and everything feels sluggish.

Flow 4: Compound command with confirmation (**send-message**)

The most complex flow. Combines typing, screenshotting, and waiting for a follow-up Signal message.



This is **stateful** in a way other commands aren't. Between **/sm** and **/c**, the daemon holds a pending draft. Three things can resolve it:

- **/c** → press Enter → "sent"
- **/x** → do nothing on KVM → text stays in box
- Timeout (default 180s) → watchdog discards the draft

If you send another **/sm** while one is pending on the same target, you get "draft already pending, reply confirm/cancel"

first" — the system refuses to overlap to keep the state coherent.

Component-by-component

signal-cli-rest-api (Docker container)

A wrapper around `signal-cli` that exposes its features as HTTP/WebSocket endpoints. Linked to your Signal account as a secondary device (like Signal Desktop). Persists keys in a Docker volume.

- Receives encrypted Signal messages and exposes them via WebSocket (`/v1/receive/<number>`)
- Sends messages via HTTP POST (`/v2/send`)
- Runs in `json-rpc` mode (the WebSocket-receive variant) — `normal` mode doesn't support live receive

signal_to_kvm.py (the daemon)

A single Python process running an asyncio event loop with three concurrent coroutines:

Coroutine	What it does
<code>receive_loop</code>	Subscribes to signal-cli WebSocket, dispatches incoming text
<code>pending_watchdog</code>	Every 5s, expires any drafts older than the timeout
HTTP server (aiohttp)	Serves <code>/poll</code> , <code>/screenshot</code> , <code>/notifications</code> to userscripts

State lives in a single `BridgeState` dataclass:

- Per-target draft state machine (`DraftState` enum)
- Notifications enabled flag (per target)
- Pending screenshot requests (request IDs → `asyncio.Event` + data)
- Counters and last-command timing for `/status`

Commands are registered via `register_command(prefix, handler, ..., aliases=...)`. The dispatcher matches longest-prefix-first, parses an optional `-target` suffix, and routes to the handler with the resolved target name.

glkvm-watcher.user.js (the userscript)

Runs in each KVM Chrome tab. Two responsibilities:

1. Watcher: samples a configurable region of the `<video>` element every ~1.5s, counts pixels matching a red profile (R high, G/B low), and fires a Signal alert when count exceeds threshold. Has its own Signal config (it doesn't go through the daemon for this).

2. Daemon bridge companion: polls the daemon's `/poll?id=<target>` every 2s. The poll response contains pending screenshot request IDs for that target plus the daemon's current `notifications_enabled` flag for that target. The userscript fulfils screenshot requests (capture from `<video>` , POST the PNG back) and reconciles its watcher state with the daemon's flag.

The userscript auto-detects its target from `location.hostname` (looks for "mac"/"win" substrings) and falls back to manual config in the panel.

run.sh (the launcher)

Foreground script. Sources `signal-to-kvm.env` , probes each configured KVM for reachability, prints a checklist with the steps you need to do manually (open browsers, start Mumble, verify keyboard layout), then `exec` s the Python daemon so Ctrl+C stops it cleanly.

setup.sh (the first-time helper)

Doesn't change anything. Prints a checklist with ✓/○ markers showing which prerequisites are done. Useful when reinstalling or moving to a new Mac because you don't have to remember which step you're on.

Why this architecture

A few choices that aren't obvious from the code alone.

Why a Python daemon, not a userscript-only solution?

Early in development, the userscript handled everything: red-watch, Signal sending, keystroke synthesis (via fake DOM events). It worked for alerts but **not for sending keystrokes** — the GLKVM web UI ignores `clientX` / `clientY` on synthetic mouse events and the WebSocket frames are partially encrypted.

Once we discovered the GLKVM exposes a REST API for HID events (`/api/hid/events/send_key` , `/api/hid/print`), the right design became:

- Userscript does what only the browser can do (read video pixels, take screenshots)
- Daemon does what only a long-running process can do (subscribe to Signal, hold state, hit the HID API authenticated)

Why one process for multiple targets, not one per target?

Originally we planned two scripts (`signal_to_kvm_mac.py` and `signal_to_kvm_win.py`). The problem: both would subscribe to the same Signal stream. When you send `/sm "hi"` , both would type — disaster.

A single process with explicit target routing solves this. The trade-off is that a bug in target Mac's command code could crash both targets. We accept that for the much simpler routing model.

Why default target + per-command suffix, not strict prefix on every command?

Tested usage: "rarely both at once." Most days, only one laptop is on. If every command required a `-mac` suffix even in single-target sessions, the ergonomic cost would dwarf the safety benefit.

Default target + override gives short commands for the common case and explicit routing when needed. `/st` always shows the current default so you can sanity-check before a sensitive command.

Why a polling bridge (`/poll` every 2s) instead of WebSocket push?

Considered. Rejected for simplicity:

- 2-second latency is fine — the only thing that polls is screenshot requests, and a 2s delay on a screenshot is unnoticeable
- WebSocket would require reconnection handling, heartbeats, lifecycle management
- The poll response carries multiple things (pending requests, notification flag, daemon status) — they ride for free on each poll

If we needed sub-second response (e.g., real-time mouse forwarding), WebSocket would be worth it. We don't.

Why no auto-start service?

Tried it. Built launchd plists, the whole nine yards. Came back to: "I use this maybe once or twice a week, why is it running 24/7?"

A foreground script you launch when you sit down gives:

- Real-time visibility (logs in your terminal)
- Clean shutdown (Ctrl+C → SIGINT → asyncio shutdown)
- No dead daemon to debug when something breaks
- A checklist printed every launch so you don't forget the setup steps

The cost is one extra command at the start of a session. For occasional use, this is the right trade.

Operational gotchas

Things that have bitten me during development. Worth knowing before they bite you again.

"Commands accepted but nothing happens"

The KVM web UI tab must be open and streaming for the GLKVM's USB HID gadget to be mounted on the host PC. If the tab is closed, `/api/hid/*` returns 200 ("ok, queued"), but nothing actually reaches the laptop. Symptom: HTTP responses look fine, cursor doesn't move, no keystrokes land.

Fix: keep the KVM Chrome tab open and at least somewhat visible.

"Screenshots time out"

Same root cause, different symptom. The userscript captures screenshots from the `<video>` element. If the tab is closed (or aggressively backgrounded by macOS), the userscript isn't polling. The daemon's screenshot request sits unfulfilled and times out at 8s.

Fix: keep both Chrome windows visible. On a headless Mac Mini, just don't sleep the display.

"Accents come out wrong"

The dead-key translation (`é` → `'e`) only works if the remote PC's input source is **U.S. International - PC**. If it's on plain US or French AZERTY, the apostrophe doesn't act as a dead key and you get `'e` literally.

Fix: set the remote PC's input source manually. There's no API to query or set it from the bridge.

"Cmd+E goes nowhere"

Teams shortcuts only work if Teams has focus. If you `/ss away` while Spotlight or another app is focused, the `/away` gets typed into whatever has focus.

There's no easy fix without OS-level scripting (which we deliberately avoided). Mitigation: make a habit of clicking into Teams on the remote screen before sending commands. Worst case, the screenshot-confirm step on `/sm` shows you where it landed.

"Signal WS error: server rejected WebSocket connection: HTTP 200"

The `signal-cli-rest-api` container is in `normal` or `native` mode instead of `json-rpc`. Only `json-rpc` mode supports WebSocket receive.

Fix: `docker rm -f signal-api` then re-create with `-e MODE=json-rpc`. The volume keeps the linked-device registration, no re-link needed.

"Daemon won't reconnect to Signal after restarting the container"

The daemon's reconnect logic has exponential backoff (2s → 60s). If you stop and start the container, the next reconnect can take up to a minute. Just wait, or restart the daemon too.

Both browser windows show the same target

The userscript's storage is per-Chrome-profile, not per-tab. If both KVM URLs are loaded in the same Chrome profile, both userscript instances share the same `targetId` config value — whichever you set last wins.

Fix: launch each Chrome window with `--user-data-dir` pointing at a separate profile directory. The hostname auto-detect handles this for you if the URLs have "mac" or "win" in them.

Extending the system

Common changes you might make later.

Adding a new command

Three steps. Example: a command `set-volume up|down|mute` that sends media-key keystrokes.

```

async def cmd_set_volume(state, target_name, payload):
    cfg = state.config
    t = state.targets[target_name].target
    arg = _strip_quotes(payload).strip().lower()

    key_map = {"up": "AudioVolumeUp", "down": "AudioVolumeDown", "mute": "AudioVolumeMute"}
    if arg not in key_map:
        await signal_send(cfg, "△ set-volume: use up|down|mute")
        return

    ok, info = await kvm_tap(t, key_map[arg])
    if ok:
        await signal_send(cfg, f"⦿ [{t.display_name}] volume {arg}")
    else:
        await signal_send(cfg, f"✕ {info}")

register_command(
    "set-volume",
    cmd_set_volume,
    "set-volume up|down|mute — adjust system volume.",
    aliases=["/sv"],
)

```

That's it. Help auto-includes it. Dispatcher auto-routes it. Target suffix auto-works. No other changes anywhere.

Adding a new platform target

Two steps. Example: a Linux target.

1. Add to `_PLATFORM_RECIPES` in the daemon:

```

"linux": {
    "display_name": "Linux",
    "shortcut_modifier": "ControlLeft",      # Teams Linux uses Ctrl
    "layout_combo": (["MetaLeft"], "Space"), # Adjust per WM
},

```

2. Add env vars: `LINUX_KVM_BASE`, `LINUX_KVM_PASSWD`, etc., and add `linux` to `KVM_TARGETS`.
3. Add a hostname keyword to the userscript's auto-detect:

```

{ keys: ['linux', 'lnx'], id: 'linux' },

```

The dispatcher and command code generalise — they pull modifier and combo from `state.targets[target_name].target`, never hardcoding "Cmd" or "Ctrl".

Changing a keystroke recipe

The recipes live in three places, ordered from "most likely to change" to "least":

1. **Per-command timing constants** in the env file (`SEARCH_AFTER_FOCUS_MS`, etc.) — change these if Teams reacts slower/faster on your machine.
2. **Per-platform modifier and layout combo** in `_PLATFORM_RECIPES` — change these if Microsoft changes Teams shortcuts.
3. **Command-specific sequences** in each `cmd_*` function — change these if the command's flow needs to change (e.g., different number of Tab presses).

Modifying the dead-key map

`_DEADKEY_MAP` in the daemon. Add a row for each char you want translated. Just remember the layout-dependence: it only works on US-International.

Glossary

Bridge — the userscript polling/screenshot channel between daemon and Chrome tabs. Not to be confused with the daemon itself, which is sometimes called "the bridge" colloquially.

Dead key — a key that doesn't produce a character on its own but composes with the next character. On US-International, `'` is a dead key: `'` followed by `e` produces `é`.

HID — Human Interface Device. The USB protocol for keyboards and mice. The GLKVM emulates a USB HID device to

the host PC.

Note to Self — Signal's name for the conversation you have with your own number. Messages there are E2EE between your devices.

Note-to-Self filter — the daemon ignores any Signal message whose sender isn't your own number. So no one else can send commands even if they somehow got into your account.

Pending draft — the state between `/sm` (text typed, awaiting your review) and `/c` or `/x` (resolved). Has a 3-minute timeout.

Reachability probe — `run.sh` checks each KVM's `/api/hid` endpoint at startup, marks it ✓ (responded) or ✗ (no response). Doesn't verify credentials, just connectivity.

Target — a configured remote machine (mac, win). Has its own KVM URL, credentials, keymap, and platform recipe.

Target id — the short name (`mac` , `win`) used as a suffix on commands and as the `?id=` parameter on userscript polls.

Userscript — the JavaScript file run by Tampermonkey in each Chrome tab. Two userscript instances run simultaneously, one per KVM window, identified by different target ids.

Watcher — the userscript's pixel-sampling background loop. Looks for red pixels in a configured zone, fires Signal alerts when triggered.